

# Attendance Management System

فارس نال او روضح ل ا قراد ا م اطن — Project README, v1.0.0

حضور وانصراف الموظفين، مبني بلغة (Multi-Company / Multi-Tenant) نظام سطح مكتب متعدد الشركات ودعم كامل للأجهزة البيومترية RTL، مع واجهة عربية SQLAlchemy وPySide6 (Qt6) باستخدام Python لإدارة (ZKTeco و Hikvision) افتراضية.

Built for organizations that need to manage attendance across an unlimited number of independent companies from a single installation, with full data isolation between tenants, biometric device integration, and Arabic-first bilingual reporting.

## المحتويات/ Contents

- نظرة عامة / Overview
- الميزات الرئيسية / Key Features
- البنية المعمارية / Architecture
- حزمة التقنيات / Tech Stack
- هيكل المشروع / Project Structure
- التثبيت والتشغيل / Installation & Running
- متغيرات البيئة / Environment Variables
- أول تشغيل / First Run
- نظام الترخيص / Licensing System
- النسخ الاحتياطي / Backup & Restore
- الأجهزة البيومترية المدعومة / Supported Devices
- الاختبار / Testing
- الترخيص / License

## نظرة عامة / Overview

من تسجيل الموظفين وأجهزة البصمة، مروراً بحساب الحضور اليومي: يوفر النظام إدارة كاملة لدورة حياة الحضور من بصمات الأجهزة أو الإدخال اليدوي، وصولاً إلى تقارير جاهزة للتصدير بصيغ (تأخير، انصراف مبكر، وقت إضافي) في الكود — بياناتها، مستخدموها، أدوارها، أقسامها، موظفوها، أجهزتها، وإعداداتها لا تشارك ولا تتقاطع أبداً مع شركة أخرى. إضافة شركة جديدة لا تتطلب أي تعديل.

أخرى، رغم أنها تعمل جميعاً فوق قاعدة بيانات واحدة: كل شركة مسجلة في النظام معزولة بالكامل عن غيرها في الكود — بياناتها، مستخدموها، أدوارها، أقسامها، موظفوها، أجهزتها، وإعداداتها لا تشارك ولا تتقاطع أبداً مع شركة أخرى. إضافة شركة جديدة لا تتطلب أي تعديل.

The system covers the full attendance lifecycle: enrolling employees and biometric devices, automatically computing daily attendance (late arrivals, early leaves, overtime) from device punches or manual entry, and exporting ready-to-share reports in Excel, PDF, and CSV. Every company is fully data-isolated from every other, even though they all run on one shared database — onboarding a new tenant requires no code changes.

## الميزات الرئيسية / Key Features

- طبقة المستودعات company\_id عزل كامل للبيانات بين الشركات عبر (Multi-Tenant): تعدد الشركات خاصة بكل شركة (Roles)، مع أدوار (Repository layer) على مستوى
- الموظفون النشطون، الأقسام، حالة الأجهزة، وملخص حضور اليوم — إحصائيات لحظية: لوحة تحكم (إجازة/غائب/متأخر/حاضر).
- موظف QR، توليد رمز (شجرة قابلة لإعادة الترتيب) بحث، تصنيف هرمي للأقسام: إدارة الموظفين والأقسام . وباركود تلقائي لكل
- استيراد بصمات من الأجهزة، إدخال يدوي، واحتساب آلي لحالة اليوم: الحضور والانصراف . بالتوقيت المحلي للشركة (عطلة/إجازة/غياب/متأخر/حاضر)
- بيانات الموظفين إلى الجهاز Hikvision (ISAPI) و ZKTeco (TCP/UDP) دعم بروتوكولات: الأجهزة البيومترية .، اختبار اتصال، مزامنة سجلات، ودفع
- (ملخص حضور، حسب الموظف، حسب القسم، المتأخرون، الوقت الإضافي، الغياب) أنواع تقارير 6: التقارير PDF. في ملفات RTL بثلاث صيغ تصدير، مع دعم كامل للنص العربي
- كامل (Permission catalog) أدوار قابلة للتخصيص لكل شركة، صلاحيات دقيقة: المستخدمين والصلاحيات . لكل عملية حساسة (Audit Log)، تدقيق
- /، ونسخ احتياطي (المنطقة الزمنية، صيغة التاريخ، العملة) ملف تعريف الشركة، تفضيلات العرض: الإعدادات (WAL متوافق مع وضع) استعادة آمن لقاعدة البيانات
- وسمة بصرية - ساعة، أرقام عربية 24، نظام DD/MM/YYYY افتراضي، تنسيق تاريخ RTL: واجهة عربية أولا Fluent Design. داكنة مستوحاة من / فاتحة (Theme) هندية اختيارية،
- فاشلة متكررة، انتهاء الجلسة عند عدم النشاط، وسجل تدقيق شامل (bcrypt) كلمات مرور مجزأة: أمان .، قفل الحساب بعد محاولات

## Architecture / البنية المعمارية

: وفصل صارم بين الطبقات Repository Pattern مع Clean Architecture النظام مبني وفق مبادئ

|               |                                          |  |
|---------------|------------------------------------------|--|
| ui/           | PySide6 windows/pages — Qt signals only  |  |
| controllers/  | Session-per-operation, ORM → dict        |  |
| services/     | Business logic, validation, audit trail  |  |
| repositories/ | Data access — the tenant-isolation point |  |
| models/       | SQLAlchemy ORM (declarative, typed)      |  |

↑ ↓ database/ (SQLAlchemy engine) devices/ (ZKTeco/Hikvision)

- يضيف — هو نقطة الإنفاذ الوحيدة CompanyScopedRepository (Tenant Isolation): العزل بين الشركات الشرط "تنسى" تلقائيا، ولا يمكن لخدمة أن ... WHERE company\_id = أي استعلام عبر مستودع مقيد بشركة هذا .
- كائنات (session\_scope()) كل عملية تنفذ داخل جلسة قاعدة بيانات مستقلة: (Controllers) نمط التحكم قد يصبح منفصلا SQLAlchemy لا تلمس الواجهة أبدا كائن — قبل إغلاق الجلسة dict إلى ORM، وتحول (Detached).
- مطابقة عبر عامة UUID معرفات و (Optimistic Locking) القفل التفاؤلي و (Soft Delete) الحذف الناعم Mixins مشتركة في models/base.py.

## Tech Stack / حزمة التقنيات

| الطبقة               | التقنية                                                           |
|----------------------|-------------------------------------------------------------------|
| الواجهة              | PySide6 (Qt6)                                                     |
| ORM / قاعدة البيانات | SQLAlchemy 2.x (SQLite افتراضيا, PostgreSQL/MySQL عبر DB_DIALECT) |
| الأمان               | bcrypt, HMAC-SHA256                                               |
| السجلات              | Loguru                                                            |
| التقارير             | openpyxl (Excel), ReportLab (PDF), CSV built-in                   |
| النص العربي في PDF   | arabic-reshaper + python-bidi                                     |
| الباركود/QR رموز     | qrcode, python-barcode                                            |
| الأجهزة البيومترية   | pyzk (ZKTeco), requests (Hikvision ISAPI)                         |
| الإعدادات            | python-dotenv                                                     |

## Project Structure / هيكل المشروع

```
attendance_system/
├── main.py          # نقطة الدخول (Composition Root)
├── config.py        # مصدر واحد للحقيقة (الإعدادات المركزية)
├── requirements.txt
├── database/
│   └── database.py  # وإدارة الجلسات SQLAlchemy محرك
├── models/          # ( نموذج 17 ) ORM نماذج
├── repositories/    # نقطة عزل الشركات) طبقة الوصول للبيانات
├── services/        # منطق الأعمال والتحقق والتدقيق
├── devices/         # (ZKTeco/Hikvision) موصلات الأجهزة البيومترية
├── controllers/     # (إشارات) الخدمات ↔ جسر الواجهة
├── ui/              # PySide6 نوافذ وشاشات
│   ├── theme.py     # (داكن/فاتح) السمة البصرية
│   ├── widgets.py   # عناصر مشتركة
│   ├── login_window.py
│   ├── main_window.py
│   └── ...          # . لوحة التحكم، الموظفون، الحضور، إلخ
├── utils/           # i18n، الأمان، السجلات، QR، التصدير
└── assets/          # الخطوط والأيقونات والترجمات
```

## Installation & Running / التثبيت والتشغيل

### Prerequisites / المتطلبات

- Python 3.13+
- الافتراضي SQLite إن أردت تجاوز MySQL أو PostgreSQL (اختياري)

### Setup / خطوات التثبيت

```
cd attendance_system
```

```
# إنشاء بيئة افتراضية / Create a virtual environment
```

```
python3 -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate

# تثبيت الاعتماديات / Install dependencies
pip install -r requirements.txt

# Optionally configure via .env
# القيم الافتراضية كافية للتشغيل المحلي دون أي تعديل
cp .env.example .env

# تشغيل التطبيق / Run the application
python main.py
```

- : تلقائيا بـmain.py عند أول تشغيل، يقوم
- 1 (إلخ, logs/, data/)إنشاء مجلدات التشغيل
  - 2.تهيئة قاعدة البيانات وإنشاء جميع الجداول
  - 3 . إن لم تكن موجودة(Permission catalog)تهيئة قائمة الصلاحيات العامة
  - 4 .عرض شاشة تسجيل الدخول

## Environment Variables /متغيرات البيئة

(. هو مصدر الحقيقة الكاملconfig.py)جميع المتغيرات اختيارية ولها قيم افتراضية معقولة للتطوير المحلي

|                         | الافتراضي          | الوصف                          |              |            |
|-------------------------|--------------------|--------------------------------|--------------|------------|
|                         | production         | development \                  | testing \    | production |
|                         | sqlite             | sqlite \                       | postgresql \ | mysql      |
|                         | data/attendance.db | SQLiteمسار ملف                 |              |            |
| PASSWORD                | —                  | PostgreSQL/MySQLإعدادات        |              |            |
| OUNDS                   | 12                 | تكلفة تجزئة كلمة المرور        |              |            |
| TIMEOUT_MI              | 30                 | مهلة الخمول قبل انتهاء الجلسة  |              |            |
| N_ATTEMPT               | 5                  | محاولات الدخول قبل القفل       |              |            |
| CKOUT_MIN               | 15                 | مدة قفل الحساب                 |              |            |
|                         | light              | light \                        | dark         |            |
| ED /<br>OURS /<br>COUNT | true / 24 / 14     | سياسة النسخ الاحتياطي التلقائي |              |            |
| T /<br>ORT              | 4370 / 80          | المنافذ الافتراضية للأجهزة     |              |            |
|                         | INFO               | مستوى التسجيل                  |              |            |

## First Run / أول تشغيل

استخدام واجهة برمجية مباشرة. لا توجد شركة أو مستخدم افتراضي مجهز مسبقاً؛ يبدأ النظام بقائمة شركات فارغة (في نسخة لاحقة "تسجيل شركة جديدة" حتى تتوفر شاشة) لإنشاء أول شركة ومستخدم مدير يمكن:

```
from database.database import get_database
get_database().initialize()

with get_database().session_scope() as session:
    from services.company_service import CompanyService
    from services.user_service import UserService
    from repositories.role_repository import RoleRepository
    from models.enums import UserRole

    company = CompanyService(session).create_company(name="الشركة")
    admin_role = next(
        r for r in RoleRepository(session, company_id=company.id).list_all()
        if r.code == UserRole.SYSTEM_ADMIN.value
    )
    UserService(session, company_id=company.id).create_user(
        username="admin",
        full_name="مدير النظام",
        password="ChangeMe123!",
        role_id=admin_role.id,
    )
```

يمكن تسجيل الدخول من التطبيق باسم المستخدم وكلمة المرور أعلاه، واختيار الشركة من قائمة تسجيل الدخول بعدها.

## Licensing System / نظام الترخيص

وعن نظام تراخيص الشركات ضمن التطبيق (Machine-Locked License) النظام محمي بترخيص مرتبط بالجهاز اشترك كل شركة داخل النظام، لا بحماية التطبيق نفسه (models/license.py)، مستقل تماماً عن قاعدة البيانات. يتحقق من الترخيص عند كل إقلاع للتطبيق، قبل شاشة تسجيل الدخول. (، وهو خاص بحدود

The application is protected by a machine-locked license, fully independent of the database and of the in-app, per-company models/license.py (which governs each tenant's subscription limits, not the application's own protection). It is verified on every startup, before the login screen.

## How it works / كيف يعمل

- الفعلي للشبكة، تعرض في شاشة التفعيل لإرسالها إلى المورد SHA-256 بصمة (Machine ID) معرف الجهاز . مشتقة من اسم الجهاز والنظام والعنوان
- نوع الترخيص (<signature>.<payload> AMS1) Ed25519 نص موقع رقمياً بخوارزمية: مفتاح الترخيص مفتاح عام مضمن في الكود فقط.، تاريخ الانتهاء، وربطاً اختيارياً بجهاز محدد (دائم/سنوي/شهري/تجريبي)، يحدد . لا يحتاج اتصالاً بالإنترنت إطلاقاً— يتحقق التطبيق من التوقيع باستخدام
- بمفتاح مشتق من معرف الجهاز نفسه، في (Fernet) الترخيص المفعل يخزن مشفراً: التخزين المحلي data/license.dat . نسخ هذا الملف إلى جهاز آخر ينتج ملفاً لا يمكن فك تشفيره هناك — data/license.dat
- تفعل ذاتياً من داخل التطبيق دون الحاجة لمفتاح من المورد، ومرة واحدة فقط لكل جهاز 14: النسخة التجريبية . يوماً،

## In-App License Management / إدارة الترخيص من داخل التطبيق

ضمن شاشة الإعدادات الحالية، دون أي تعديل على بقية تبويباتها) الترخيص→الإعدادات بعد تسجيل الدخول، من الحالة، تاريخ التفعيل، تاريخ الانتهاء، معرف الجهاز، معرف الترخيص) عرض تفصيلي لحالة الترخيص : (تبويب جديد : مع خمسة أزرار إدارة(اسم الشركة، اسم العميل، النوع،

- لصق مفتاح جديد يحل محل الحالي:تفعيل ترخيص .
- تاريخ الانتهاء الجديد فوراً — (بما فيها المنتهية) مخصص للاشتراكات الشهرية والسنوية فقط :تجديد الترخيص . يرفض مفاتيح التجديد من نوع تجريبي أو دائم، ويظهر
- الترخيص، اسم الشركة والعميل، معرف الجهاز، وتفاصيل التفعيل JSON يحفظ ملف :تصدير طلب الترخيص الترخيص الموقع الأصلي كاملاً، وهو ما يجعل الملف قابلاً للتحقق بشكل مستقل من قبل المورد — يحتوي معرف . دون الحاجة لأي مفتاح توقيع إضافي داخل التطبيق(أو أي طرف يملك المفتاح العام) مع تضمين مفتاح
- يسمح الترخيص من هذا الجهاز فوراً، مع خيار حفظ ملف طلب النقل في نفس الخطوة:إلغاء تفعيل الترخيص . يرسل العميل هذا الملف إلى المورد، الذي يصدر بعدها مفتاحاً جديداً لجهاز مختلف.
- نسخ معرف الجهاز .

فإن منع تفعيل نفس الترخيص على جهازين في آن واحد هو إجراء تنظيمي :ملاحظة مهمة حول نقل التراخيص فوراً (المورد يراجع ملف طلب النقل قبل إصدار مفتاح جديد) بما أن النظام يعمل بالكامل دون اتصال بالإنترنت، ذلك بشكل آلي يتطلب خادم ترخيص مركزي، وهو بالضبط ما تم تجهيز البنية له مسبقاً — وليس ضماناً تشفيرياً . دون الحاجة لأي تعديل على واجهات الترخيص الحالية(انظر أدناه) تحقيق

Post-login, from Settings → License (a new tab on the existing Settings screen — no other tab was touched): a full status view plus five actions — Activate, Renew (Monthly/Yearly only, including already-expired ones; rejects Trial/Lifetime renewal keys and shows the new expiry immediately), Export License Request (a JSON file embedding the original signed key itself as independently-verifiable proof — no extra signing key needed in the app), Deactivate (clears this machine immediately, optionally exporting a transfer request in the same step for the vendor to review before issuing a replacement key elsewhere), and Copy Machine ID. Preventing simultaneous activation on two machines is, honestly, a procedural control here (vendor reviews the transfer request) rather than a real-time cryptographic guarantee — that needs the online-backend extension point described below, with zero changes required to these screens when it's added.

## Issuing License Keys / (للمورد فقط) إصدار مفاتيح الترخيص (Vendor-Only)

أداة سطر أوامر منفصلة تماماً عن التطبيق، لا تستدعى منه أبداً:

# (احتفظ بالمفتاح الخاص بمكان آمن خارج المستودع) توليد زوج مفاتيح التوقيع : مرة واحدة فقط #

```
python -m licensing.license_generator generate-keypair \
  --private-key-out /secure/location/private_key.pem \
  --public-key-out /tmp/public_key.pem
# licensing/keys.py -> PUBLIC_KEY_PEM ثم انسخ محتوى المفتاح العام إلى
```

# (اختياري--company إصدار مفتاح سنوي مرتبط بجهاز عميل محدد

```
python -m licensing.license_generator issue \
  --private-key /secure/location/private_key.pem \
  --customer "اسم العميل" --company "اسم الشركة" \
  --type yearly \
  --machine-id <Machine-ID-الذي أرسله>
```

# (يفعل على أول جهاز يستخدمه) إصدار مفتاح دائم غير مرتبط بجهاز

```
python -m licensing.license_generator issue \
  --private-key /secure/location/private_key.pem \
  --customer "اسم العميل" \
```

--type lifetime

الخاص الذي يسمح بإصدار تراخيص صالحة لهذا التطبيق licensing/vendor/private\_key.pem: تحذير أمني  
فعليا عبر ) لا يجوز مطلقا تضمينه في التطبيق الموزع أو رفعه إلى نظام التحكم بالإصدارات . هو المفتاح  
المفتاح العام المضمن حاليا؛ تسريبه يعني إمكانية تزوير تراخيص من قبل أي طرف يحصل عليه.(.gitignore مستثنى  
فقدانه يعني عدم القدرة على إصدار تراخيص جديدة لنفس .

## Extending to an Online License Server / التوسع لخادم ترخيص عبر الإنترنت لاحقا

في (verify(key) -> LicensePayload) بسيطة LicenseBackend طبقة التحقق مبنية خلف واجهة  
يعمل بالكامل دون اتصال بالإنترنت(LocalLicenseBackend) التنفيذ الحالي . licensing/license\_service.py.  
صنف جديد يحقق نفس الواجهة وتمريضه إلى (مثل التأكد من عدم إلغاء الاشتراك) لإضافة تحقق عبر خادم لاحقا  
أي تعديل على واجهة التفعيل أو التخزين المحلي أو بقية التطبيق — LicenseService(backend=...)، يكفي إنشاء  
دون .

## Backup & Restore / النسخ الاحتياطي

واجهة . إنشاء نسخة فورية، استعراض النسخ السابقة، والاستعادة منها: النسخ الاحتياطي→الإعدادات من شاشة  
بعد إلى القرص—، وليس نسخا خام للملف WAL الآمنة مع وضع sqlite3.Connection.backup() الآلية تستخدم  
الاستعادة تستبدل جميع البيانات الحالية بشكل نهائي: تنبيه . يضمن ذلك عدم فقدان بيانات لم تكتب

## Supported Devices / الأجهزة البيومترية المدعومة

| الحقل المطلوب               | الاتصال                                 | البروتوكول        |
|-----------------------------|-----------------------------------------|-------------------|
| (اختياري) كلمة اتصال الجهاز | افتراضيا 4370، المنفذ TCP               | ZKTeco TCP/IP     |
| (اختياري) كلمة اتصال الجهاز | افتراضيا 4370، المنفذ UDP               | ZKTeco UDP        |
| المروor كلمة: المستخدم اسم  | المنفذ HTTP Digest Auth<br>افتراضيا 80، | Hikvision (ISAPI) |

السجلات آمنة عند التكرار (Employee Number) الرقم الوظيفي مطابقة سجلات الجهاز بالموظفين تتم عبر  
(لا تنشئ بصمات مكررة عند إعادة المزامنة)، ومزامنة

## Testing / الاختبار

وكل شاشة واجهة) مضمنة في المستودع؛ تم التحقق من كل طبقة (test suite) لا توجد مجموعة اختبارات آلية  
عبر سيناريوهات تشغيل حقيقية ضد قاعدة بيانات (النماذج، المستودعات، الخدمات، الأجهزة، وحدات التحكم،  
(. للواجهات QT\_QPA\_PLATFORM=offscreen بوضع ) حقيقي QApplication فعلية وSQLite يدويا أثناء التطوير

: لأغراض الاختبار(headless) لتشغيل التطبيق في بيئة بلا شاشة

QT\_QPA\_PLATFORM=offscreen python main.py

## License / الترخيص

Bitstream مرخصة بموجب (assets/fonts/DejaVuSans\*.ttf) الخطوط المرفقة . هذا مشروع داخلي مخصص  
Vera License والتوزيع التجاري . وتسمح بالاستخدام والتوزيع التجاري